Implementation of Scheduler History System Call in Linux

Project Report

Operating Systems [CS F372]

Semester II, 2025 – 2026

BITS Pilani
K K Birla Goa Campus

Course Instructor: Dr Biju K Raveendran

Submitted By

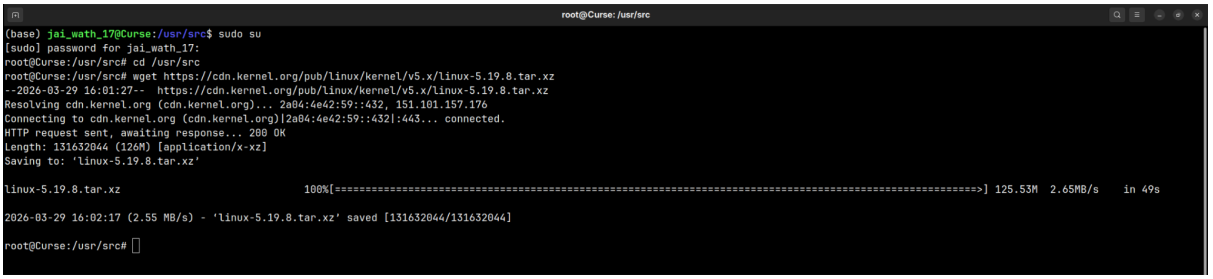
Group 36 – KernelX

Jaikumar Wath 2023A3PS0197G
Ayush Pareek 2023AAPS0609G

INDEX

1. Kernel Recompilation

1.1 Downloading Kernel Source

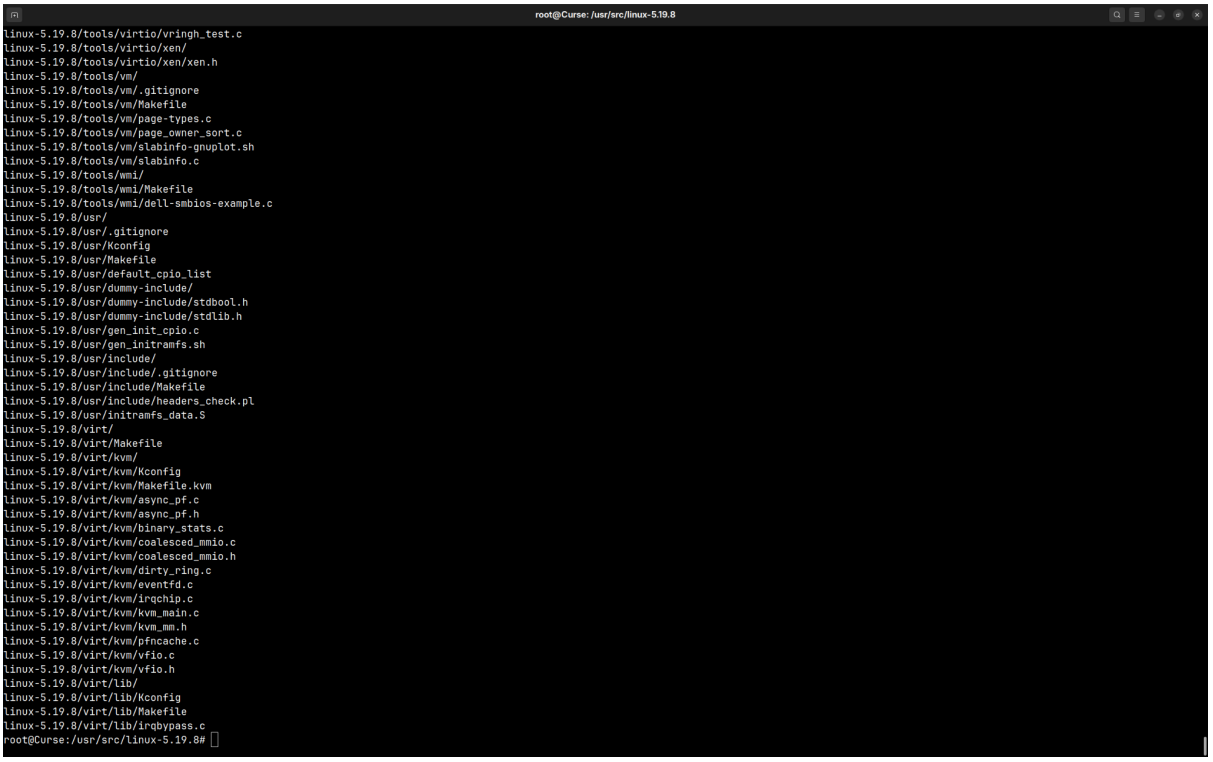


The Linux kernel source (version **5.19.8**) was downloaded using:

cd /usr/src
sudo su
wget https://cdn.kernel.org/pub/linux/kernel/v5.x/linux-5.19.8.tar.xz

The file `linux-5.19.8.tar.xz` was successfully downloaded into the `/usr/src` directory, and the working directory was set to `/usr/src.`

1.2 Extract the Kernel Source

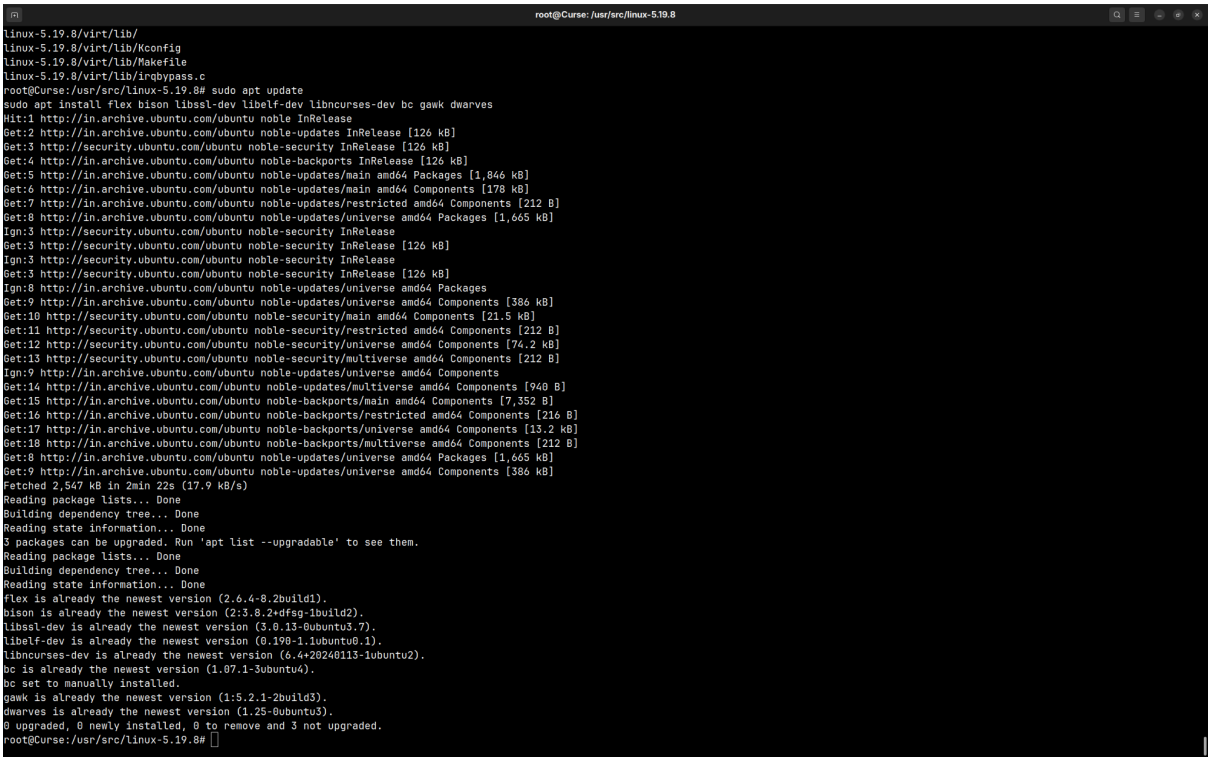


The kernel source was extracted using:

tar -xvf linux-5.19.8.tar.xz
cd linux-5.19.8

This created a directory named `linux-5.19.8` containing the kernel source code, and the working directory was changed to the extracted kernel source.

1.3 Install Required Dependencies



Before compiling the Linux kernel, it is necessary to install the required development tools and libraries. These dependencies are essential for building the kernel successfully without errors.

The following commands were executed to update the package list and install the required packages:

sudo apt update
sudo apt install flex bison libssl-dev libelf-dev bc build-essential dwarves

1.4 Kernel Configuration

```
root@Curse:/usr/src/linux-5.19.8# make olddefconfig
#
# configuration written to .config
#
root@Curse:/usr/src/linux-5.19.8#
```

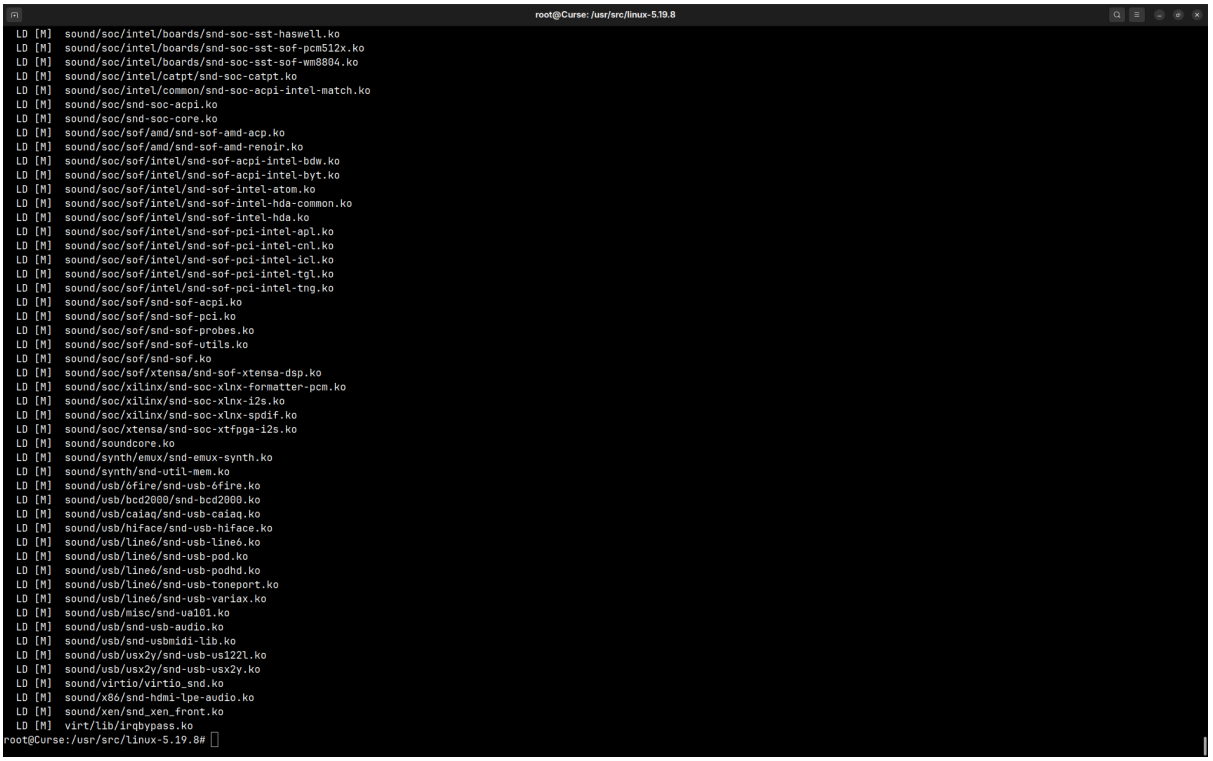
Before compiling the kernel, the configuration file must be prepared. This step ensures that the kernel is configured according to the system requirements.

The existing system configuration was reused using the following command:

make olddefconfig

This command updates the configuration file based on the current system defaults and ensures compatibility with the kernel version being compiled.

1.5 Kernel Compilation



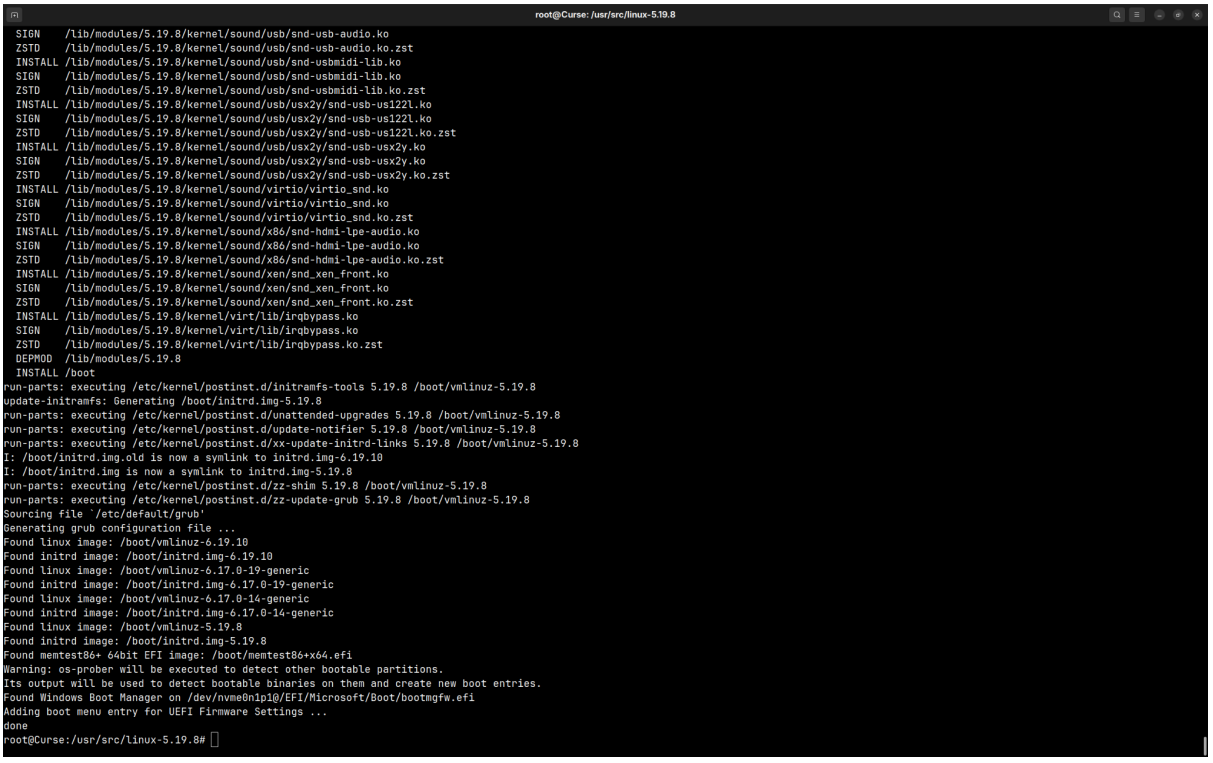
After configuring the kernel, the next step was to recompile the Linux kernel. This process builds the kernel image and all necessary modules required for system operation.

The kernel was compiled using the following command:

make -j$(nproc)

This command utilizes all available CPU cores, thereby speeding up the compilation process. During compilation, various kernel components such as drivers, modules, and core subsystems were built successfully.
The compilation process completed without issues after proper configuration.

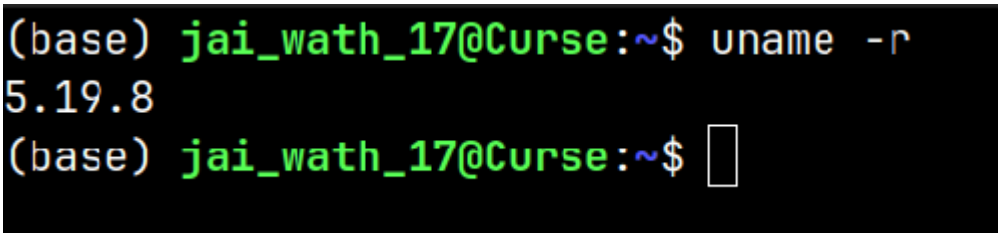
1.6 Kernel Installation & verification



After compiling the kernel, it was installed using:

make modules_install
make install

These commands install the modules in /lib/modules and the kernel in /boot.
The system also updates initramfs and GRUB to enable booting into the new kernel.

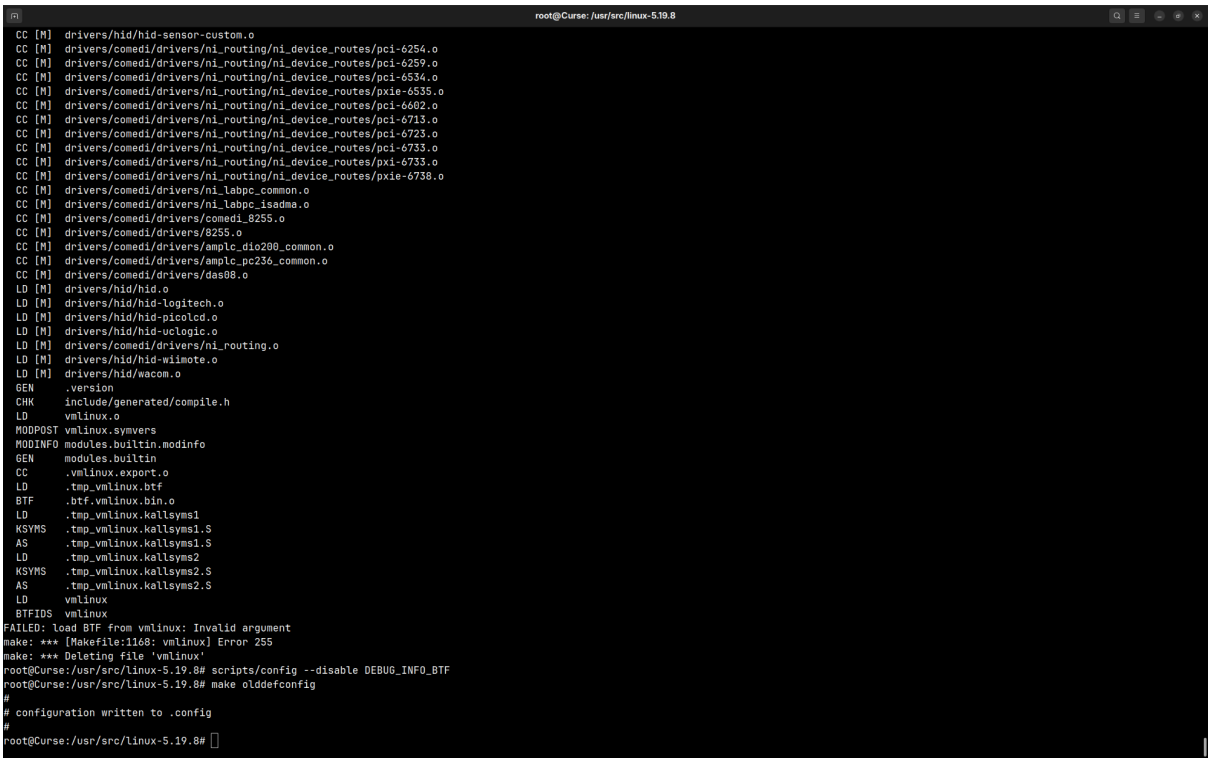


After installation, the system was rebooted to load the newly installed kernel.
The kernel version was verified using the following command:

uname -r

The output confirmed that the system is running the newly compiled kernel version.

1.7 Error Handling



During kernel compilation, an error related to BTF (BPF Type Format) was encountered: "FAILED: load BTF from vmlinux: Invalid argument"
This issue was resolved by disabling the DEBUG_INFO_BTF option using the following commands:

scripts/config --disable DEBUG_INFO_BTF
make olddefconfig

After applying this fix, the kernel compilation proceeded successfully without further errors.

2. System Call Implementation

Scheduler History System Call

The system call `get_sched_history` is implemented to track the scheduling behavior of processes in the Linux kernel. It records when a process is scheduled in and scheduled out of the CPU.

Each scheduling event stores the timestamp, CPU core, event type, and process ID. The events are maintained in a fixed-size buffer inside the kernel.

The system call takes a process ID and returns its recent scheduling history to user space.

2.1 System Call Setup and Directory Creation

A new directory `sched_syscall` was created inside the Linux kernel source to store files related to the custom system call.

cd /usr/src/linux-5.19.8
mkdir sched_syscall

```
(base) jai_wath_17@Curse:/usr/src/linux-5.19.8$ sudo su
[sudo] password for jai_wath_17:
root@Curse:/usr/src/linux-5.19.8# mkdir sched_syscall
root@Curse:/usr/src/linux-5.19.8# ls
arch   COPYING  Documentation  include  ipc      kernel    MAINTAINERS  modules.builtin          modules.order   README        scripts   System.map  virt           vmlinux.map
block  CREDITS  drivers        init     Kbuild   lib       Makefile     modules.builtin.modinfo  Module.symvers  samples       security  tools       vmlinux        vmlinux.o
certs  crypto   fs             io_uring Kconfig  LICENSES  mm           modules-only.symvers     net             sched_syscall sound     usr         vmlinux-gdb.py vmlinux.symvers
root@Curse:/usr/src/linux-5.19.8#
```

2.2 Implementation of System Call Logic

Inside the `sched_syscall` directory, the source file `sched_syscall.c` was created to implement the custom system call.

cd sched_syscall
touch sched_syscall.c

This file contains the data structure, buffer management, and logic required to record scheduling events.

Data Structure and Buffer

A structure named `sched_event` was defined to represent each scheduling event. It stores the timestamp, CPU core, event type, and process ID.

A fixed-size buffer was also created to store scheduling events inside the kernel. The buffer operates in a circular manner, ensuring that older events are overwritten when full.

Logging Scheduling Events

A function named `log_sched_event` was implemented to record scheduling events.

This function captures the current timestamp, CPU core, event type, and process ID, and stores them in the buffer. The buffer index is updated in a circular manner to continuously store recent events.

System Call Implementation

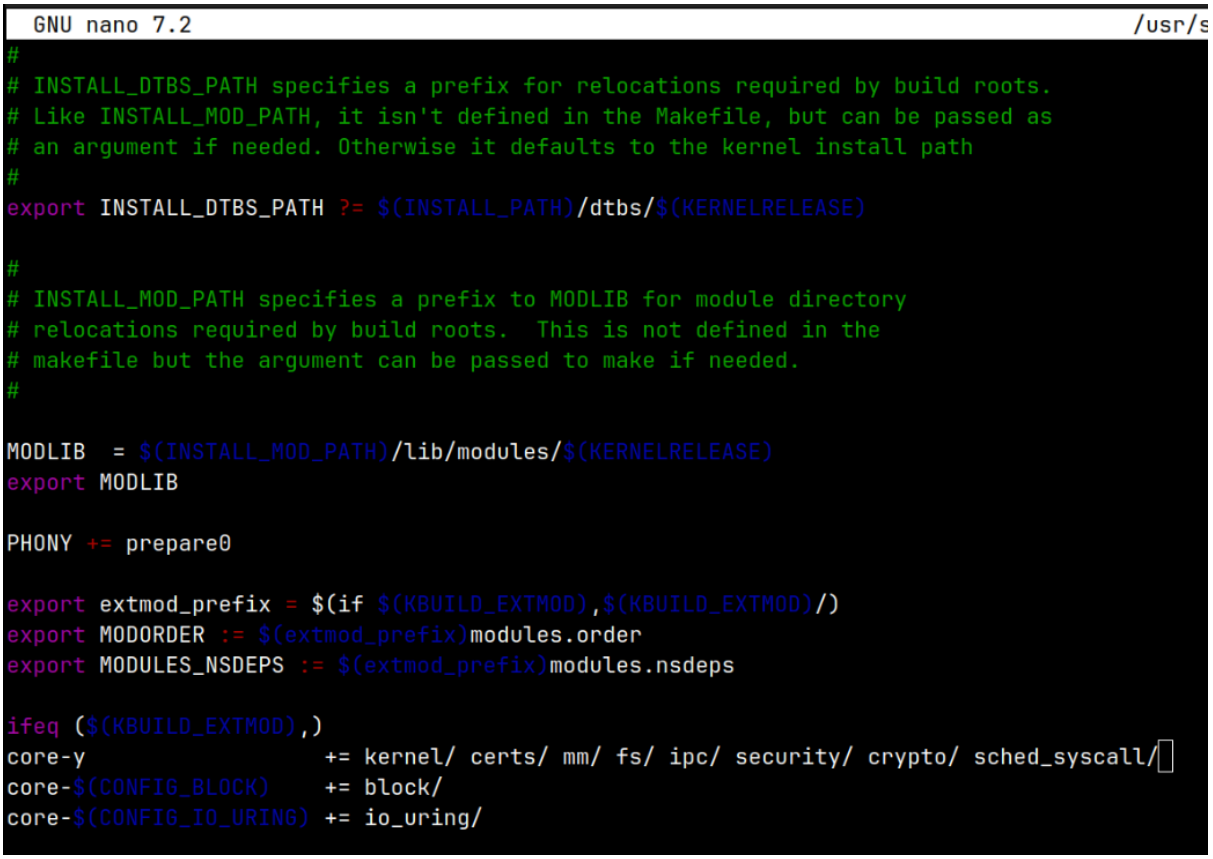
The system call `get_sched_history` was implemented using the `SYSCALL_DEFINE3` macro.

It takes a process ID, a user-space buffer, and a maximum event limit as input. The system call filters events corresponding to the given process and copies them safely to user space using `copy_to_user`. It returns the number of events retrieved.

2.3 Modifications in Kernel Source Files

To integrate the custom system call into the Linux kernel, several kernel source files were modified.

1. Makefile Modification

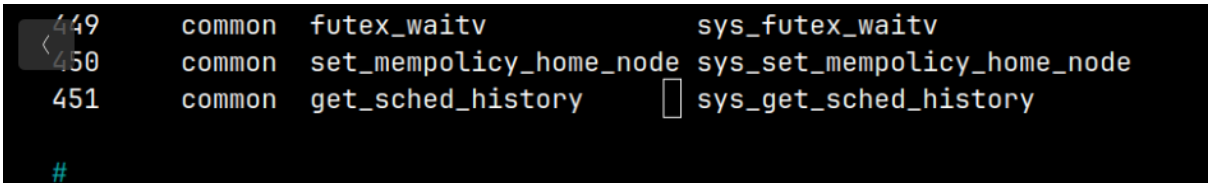


The kernel `Makefile` was updated by adding the `sched_syscall/` directory to the `core-y` list:

core-y += kernel/ certs/ mm/ fs/ ipc/ security/ crypto/ sched_syscall/

This ensures that the system call implementation is compiled as part of the kernel build.

2. System Call Table Update



The file `syscall_64.tbl` was modified to register the new system call.

A new entry was added:

451 common get_sched_history sys_get_sched_history

This assigns a unique system call number and links it to its implementation.

3. System Call Declaration

```
 GNU nano 7.2
asmlinkage long sys_ftruncate64(unsigned int fd, loff_t length);
#endif
asmlinkage long sys_fallocate(int fd, int mode, loff_t offset, loff_t len);
asmlinkage long sys_faccessat(int dfd, const char __user *filename, int mode);
asmlinkage long sys_faccessat2(int dfd, const char __user *filename, int mode,
                               int flags);
asmlinkage long sys_chdir(const char __user *filename);
asmlinkage long sys_fchdir(unsigned int fd);
asmlinkage long sys_chroot(const char __user *filename);
asmlinkage long sys_fchmod(unsigned int fd, umode_t mode);
asmlinkage long sys_fchmodat(int dfd, const char __user * filename,
                             umode_t mode);
asmlinkage long sys_fchownat(int dfd, const char __user *filename, uid_t user,
                             gid_t group, int flag);
asmlinkage long sys_fchown(unsigned int fd, uid_t user, gid_t group);
asmlinkage long sys_openat(int dfd, const char __user *filename, int flags,
                           umode_t mode);
asmlinkage long sys_openat2(int dfd, const char __user *filename,
                            struct open_how *how, size_t size);
asmlinkage long sys_close(unsigned int fd);
asmlinkage long sys_close_range(unsigned int fd, unsigned int max_fd,
                                unsigned int flags);
asmlinkage long sys_vhangup(void);
asmlinkage long sys_get_sched_history(pid_t pid,
                                      void __user *buf,
                                      int max_events);

/* fs/pipe.c */
asmlinkage long sys_pipe2(int __user *fildes, int flags);
```

The file `syscalls.h` was updated by adding the declaration of the system call:

asmlinkage long sys_get_sched_history(pid_t pid, void __user *buf, int max_events);

This allows the kernel to recognize the system call interface.

4. Scheduler Modification (core.c)

```
/*
 * context_switch - switch to the new MM and the new thread's register state.
 */
extern void log_sched_event(pid_t pid, int event);

static __always_inline struct rq *
context_switch(struct rq *rq, struct task_struct *prev,
               struct task_struct *next, struct rq_flags *rf)
{
    prepare_task_switch(rq, prev, next);

    log_sched_event(prev->pid, 1); // OUT
    log_sched_event(next->pid, 0); // IN

        /*
         * For paravirt, this is coupled with an exit in switch_to to
         * combine the page table reload and the switch backend into
         * one hypercall.
         */
        arch_start_context_switch(prev);

        /*
         * kernel -> kernel   lazy + transfer active
         *   user -> kernel   lazy + mmgrab() active
         *
```

The kernel scheduler file `core.c` was modified to log scheduling events during context switches.

A call to the logging function `log_sched_event` was added inside the `context_switch` function:

- When a process is switched out → event recorded as OUT
- When a process is switched in → event recorded as IN

This ensures that every context switch is captured and stored in the buffer.

5. System Call Makefile

```
  GNU nano 7.2
obj-y := sched_syscall.o
```

A `Makefile` was created inside the `sched_syscall` directory:

obj-y := sched_syscall.o

This ensures that the system call source file is compiled into the kernel.

3. User Space Implementation

A user-space program `test.c` was developed to invoke the `get_sched_history` system call and retrieve scheduling events of a process.

The program defines the system call number and uses the `syscall()` interface to pass the process ID, a buffer, and the maximum number of events.

It generates scheduling activity and then retrieves the recorded events from the kernel.

The retrieved data is processed and displayed in a formatted manner, showing:

- Timestamp
- CPU core
- Process ID
- Event type (IN/OUT)

```
  GNU nano 7.2
#include <stdio.h>
#include <unistd.h>
#include <sys/syscall.h>

#define __NR_get_sched_history 451
#define MAX_EVENTS 1000

struct sched_event {
    unsigned long long timestamp;
    int cpu;
    int event; // 0 = IN, 1 = OUT
    int pid;
};

int main()
{
    struct sched_event buffer[MAX_EVENTS];

    int pid = getpid();
    printf("PID: %d\n\n", pid);

    //  generate scheduling activity
    for (volatile long i = 0; i < 2000000000; i++);

    int ret = syscall(__NR_get_sched_history,
                      pid,
                      buffer,
                      MAX_EVENTS);

    if (ret <= 0) {
        printf("No events captured\n");
        return 0;
    }

    printf("%-12s %-5s %-6s %-5s\n", "Time(ns)", "CPU", "PID", "Event");

    int printed = 0;

    for (int i = 0; i < ret; i++) {
        if (buffer[i].pid == pid) {

            printf("%-12llu %-5d %-6d %-5s\n",
                   buffer[i].timestamp,
                   buffer[i].cpu,
                   buffer[i].pid,
                   buffer[i].event == 0 ? "IN" : "OUT");
```

```
            printed++;

            if (printed >= 6) break;  //  3 IN/OUT pairs
        }
    }

    return 0;
}
```

3.1 System Call Invocation

The system call is invoked using:

syscall(451, pid, buffer, MAX_EVENTS);

This returns the number of scheduling events captured for the given process.

3.2 Compiling and Executing the User program

```
(base) jai_wath_17@Curse:~/Documents/os_proj/user_space$ ./test
PID: 6912

No events captured
(base) jai_wath_17@Curse:~/Documents/os_proj/user_space$ ./test
PID: 6913

Time(ns)     CPU   PID    Event
167218202531 8     6913   OUT
167218224387 9     6913   IN
(base) jai_wath_17@Curse:~/Documents/os_proj/user_space$ ./test
PID: 6916

Time(ns)     CPU   PID    Event
169777198289 8     6916   OUT
169777203037 8     6916   IN
(base) jai_wath_17@Curse:~/Documents/os_proj/user_space$ killall yes
[1]-  Terminated              yes > /dev/null
[2]+  Terminated              yes > /dev/null
(base) jai_wath_17@Curse:~/Documents/os_proj/user_space$ yes > /dev/null &
yes > /dev/null &
yes > /dev/null &
[1] 6949
[2] 6950
[3] 6951
(base) jai_wath_17@Curse:~/Documents/os_proj/user_space$ ./test
PID: 6960

Time(ns)     CPU   PID    Event
246770201168 8     6960   OUT
246770208361 8     6960   IN
(base) jai_wath_17@Curse:~/Documents/os_proj/user_space$ ./test
PID: 6963

Time(ns)     CPU   PID    Event
249575553304 4     6963   OUT
249575558542 4     6963   IN
(base) jai_wath_17@Curse:~/Documents/os_proj/user_space$ ./test
PID: 6964

Time(ns)     CPU   PID    Event
252475933302 0     6964   OUT
252475948945 0     6964   IN
252475987984 0     6964   OUT
252476013195 0     6964   IN
252518597726 0     6964   OUT
252518612811 0     6964   IN
(base) jai_wath_17@Curse:~/Documents/os_proj/user_space$
```

When the program prints **“No events captured”**, it means that no scheduling events were recorded for that process. The system call only returns events that are logged during **context switches**, so if the process has not yet been switched in or out by the scheduler, the buffer remains empty.

The command `yes > /dev/null &` is used to generate scheduling activity. The `yes` command continuously produces output, creating a CPU-intensive process. The output is redirected to `/dev/null`, which discards it, preventing terminal clutter. The `&` runs the process in the background. This combination forces the CPU to stay busy.

Due to this increased CPU usage, the scheduler frequently performs **context switches** between processes. Each switch triggers the logging function added in the kernel, which records IN and OUT events. This is why, after running this command, scheduling events start appearing in the output.

In the final output, multiple IN and OUT events are shown for the same process along with timestamps and CPU IDs. This indicates that the process was scheduled multiple times on the CPU, demonstrating correct functioning of the system call and confirming that scheduling activity is being successfully captured.